BillPay: how the app works, and the points still open

Final-year project CCDS26-0127 | Anshika Uppal | Supervisor: Dr Chloe Gu | Integrated Bill Payment App with Smart Categorization

1. What the app does

BillPay is a mobile app for paying recurring bills (power, phone, insurance, subscriptions) in one place. A user links billers, pays through the app, and every payment is tagged with a spending category automatically, with no manual entry. The project question is whether a cheap, two-stage categorizer (a rule table for known billers plus a small trained model for unknown ones) can do this well for Singapore billers. That categorizer, and how well it is evaluated, is the main thing being assessed. The app itself is the vehicle.

2. The pieces

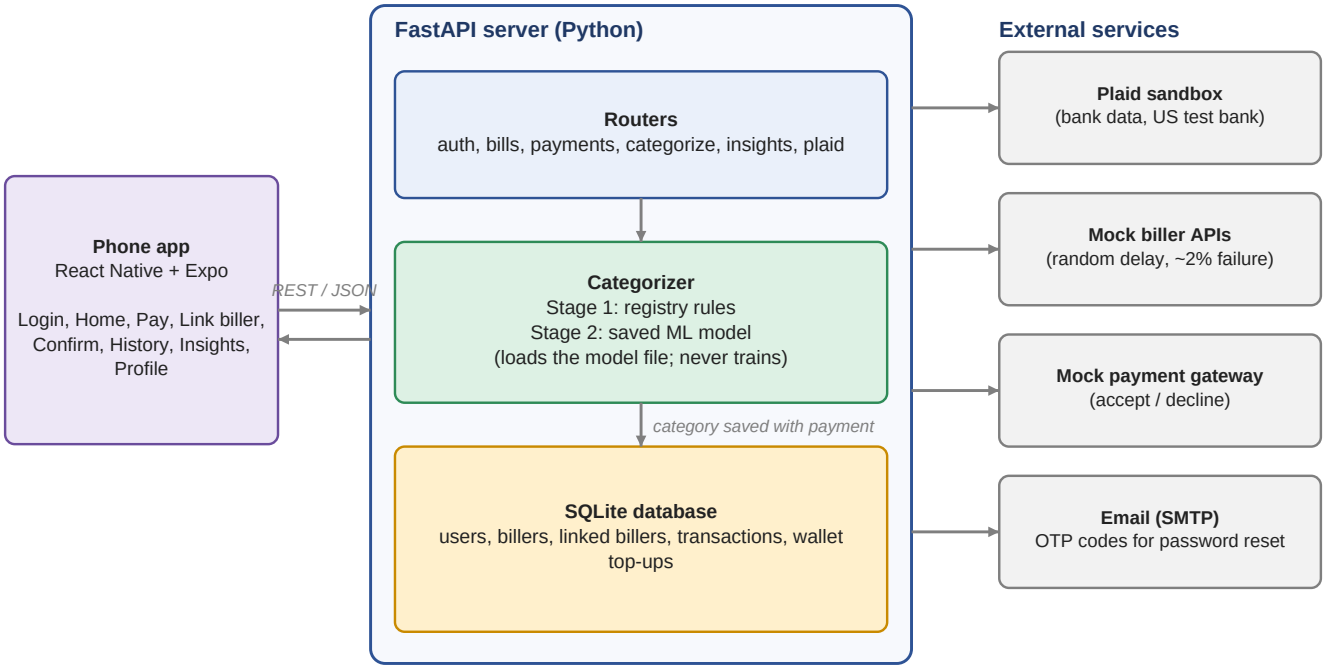


Figure 1. The phone app never touches the database; it only calls the server. The server holds the data, runs the categorizer and talks to the external services.

The user journey

Step	What the user does	What happens behind it
1 Register / log in	Email and password; OTP code for password reset	Server checks credentials and stores a hashed password. OTP codes are emailed (printed to the server console in development).
2 Link a biller	Picks SP Power, Singtel, etc. from the directory	The directory comes from the same biller registry the categorizer uses. The mock biller API returns an amount due, with random delay and occasional failure.
3 Pay	Pays from wallet or directly	Mock payment gateway accepts or declines. The categorizer tags the payment (next page). The payment is saved with its category, confidence and method.
4 See spending	History and Insights screens	Server totals spending by category and month.

3. What happens when a payment is categorised

Every payment passes through the same cascade. Stage 1 is fast and exact for billers the registry knows. Stage 2 only runs when Stage 1 finds nothing, and when it is not confident the system says so rather than guessing.

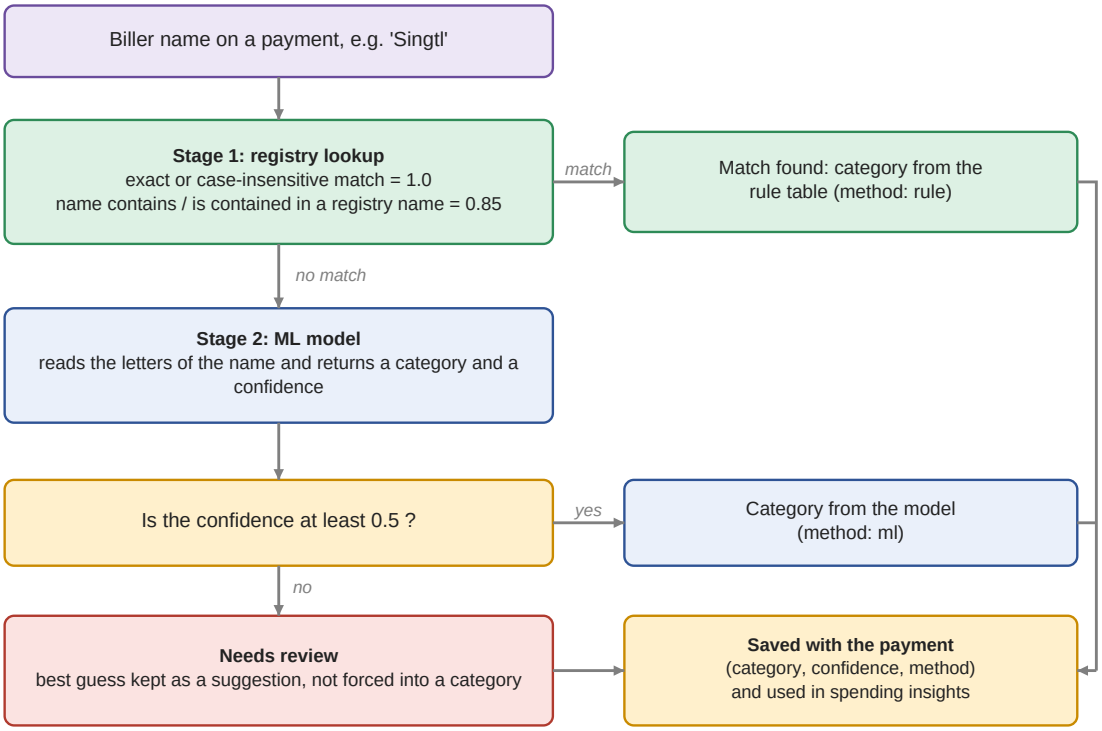


Figure 2. The categorization cascade. This is a cascade, not an ensemble: only one stage answers.

Four examples, one per outcome

Name entered	Who answers	Result	Why
Singtel	Stage 1, exact match	Telecoms, 1.0 (rule)	It is in the registry.
SP Power Pte Ltd	Stage 1, partial match	Utilities, 0.85 (rule)	Contains the registry name 'SP Power'.
Singtl (typo)	Stage 2 model	Telecoms, 0.82 (ml)	Letter chunks are close to 'Singtel'.
Unknown Merchant XYZ	Stage 2 model, unsure	Needs review, 0.34	Nothing similar; best guess (Finance) is kept as a suggestion only.

What 'confidence' means here

The model scores each of the 10 categories and the scores add up to 100%. The confidence is the share of the winning category. With 10 categories, the lowest possible top score is 10%, so a low number means 'no good match', not 'probably right'. That is why the system abstains below 0.5.

4. How the model was built

The registry alone (51 billers) is too small to train on, so the training data is generated from it. To avoid hand-chosen variations, the generator follows transformation types reported in published research (the supervisor's request on human bias). Training happens offline in a notebook; the server only loads the saved result.

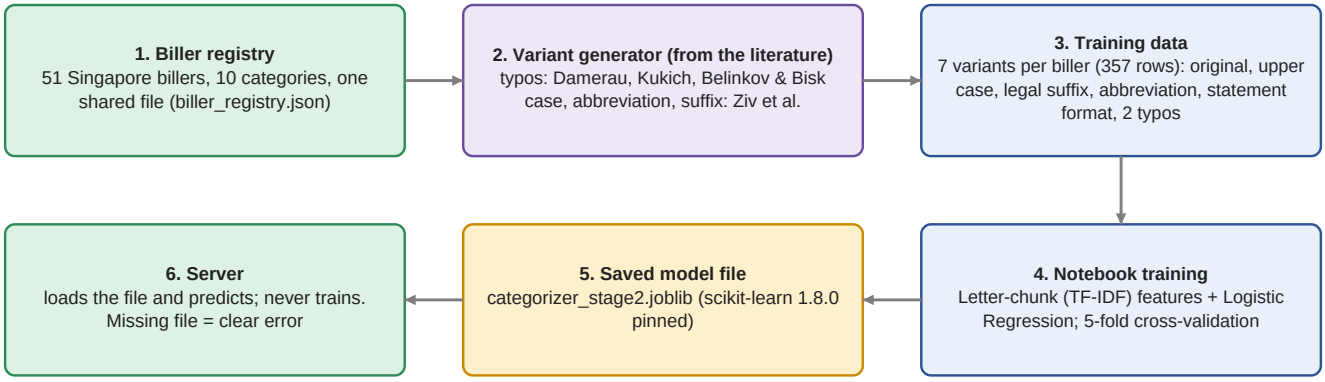
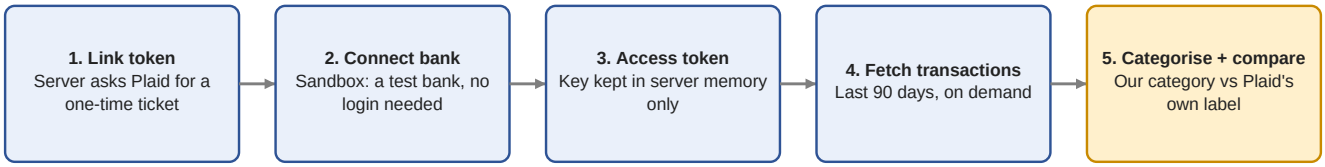


Figure 3. Training pipeline. The registry file is shared by Stage 1 (rules), the app's biller list and the training notebook, so they cannot drift apart.

5. How it was tested against Plaid

Plaid is a US service that lets apps read bank transactions. Here it is used only as an independent check: the same transactions are categorised by BillPay and compared with Plaid's own category labels.



Limit: Plaid covers the US, Canada and Europe, not Singapore, so the sandbox returns US merchants (Uber, KFC, United Airlines...).
Only transactions whose Plaid label maps onto our categories can be compared (27 of 48 in the first run).

Figure 4. Plaid sandbox flow. In the sandbox, one shortcut call replaces steps 1 to 3. The phone-app screen for it is built but not yet wired into the app.

6. Results so far, honestly

Test	Result	How to read it
Generated data, held-out part and 5-fold cross-validation	About 98%	Optimistic. Variants of the same billers sit in both the training and test parts, so the model has seen every biller.
8 quick names the model had not seen	3 right, 1 confidently wrong, 4 'Needs review'	Handles typos (Singtl) and names with an industry word (Delta Airlines to Travel). Wrong on Starbucks (Telecoms, 0.69, because it looks like StarHub). 'Singapore Power' was pulled toward Travel by 'Singapore Airlines'. Informal test only.
Plaid sandbox, 48 transactions over 90 days	27 comparable; 6 matched	All 6 matches were KFC and McDonald's, which Stage 1 caught because they are in the registry. The model itself got none of the comparable rows right: United Airlines was Travel at 0.47 (just under the cut-off), Uber needed review, Starbucks was wrong.

Not yet true or not done: no automated tests; code is not yet on GitHub (one commit only); OTP email is not configured (codes print to the console); the bank-link screen is not wired into the app; Google login does not verify the token; the app's server address is hard-coded.

7. Points to close: your input needed

Each row has my current lean. Please add your answer or a better option in the last column.

#	Question	Options	My lean	Your answer
1	Plaid has no Singapore coverage. What role should it play?	Secondary check only; drop it; keep as headline benchmark	Secondary check, reported with how many rows were comparable	
2	What is the main accuracy measure for the unseen-biller claim (target 70%)?	Hold out whole billers; hand-labelled Singapore statement set; both	Hold out whole billers as the main number, plus a hand-labelled set	
3	The model misreads shared words (Singapore Power as Travel) and has no industry-word knowledge. Fix how?	Add industry words (Power, Airlines, Hotel) as training rows; keyword rules; neither	Take the words from the public biller lists (not my own picks) and test on names I have not looked at	
4	Brand-only names (Uber, Starbucks) are not in the registry and carry no industry word. How to categorise?	Leave as 'Needs review' for the user to confirm; add an external merchant list; add more brands	'Needs review' plus user confirmation; list brands later	
5	Cut-off for 'Needs review' is a fixed 0.5. United Airlines was right at 0.47.	Keep 0.5; lower it; also use the gap to the runner-up	Add a margin rule, with the numbers chosen by cross-validation, not by Plaid	
6	Registry: only 45 of 51 are bills; Food and Travel were added so common merchants get a label. Scope?	Keep bills only; keep 10 categories; grow to 80-100 verified from DBS / HSBC / UOB lists	Keep 10 categories and verify against the public lists as it grows	
7	Plan wording: performance target says 2 seconds in one place and 5 in another; some section references are wrong.	2 seconds; 5 seconds	2 seconds (mock APIs add about 1 second)	
8	Housekeeping before the next report: tests, GitHub, email OTP, bank-link screen.	Do all; do only tests and GitHub	Tests and GitHub first	